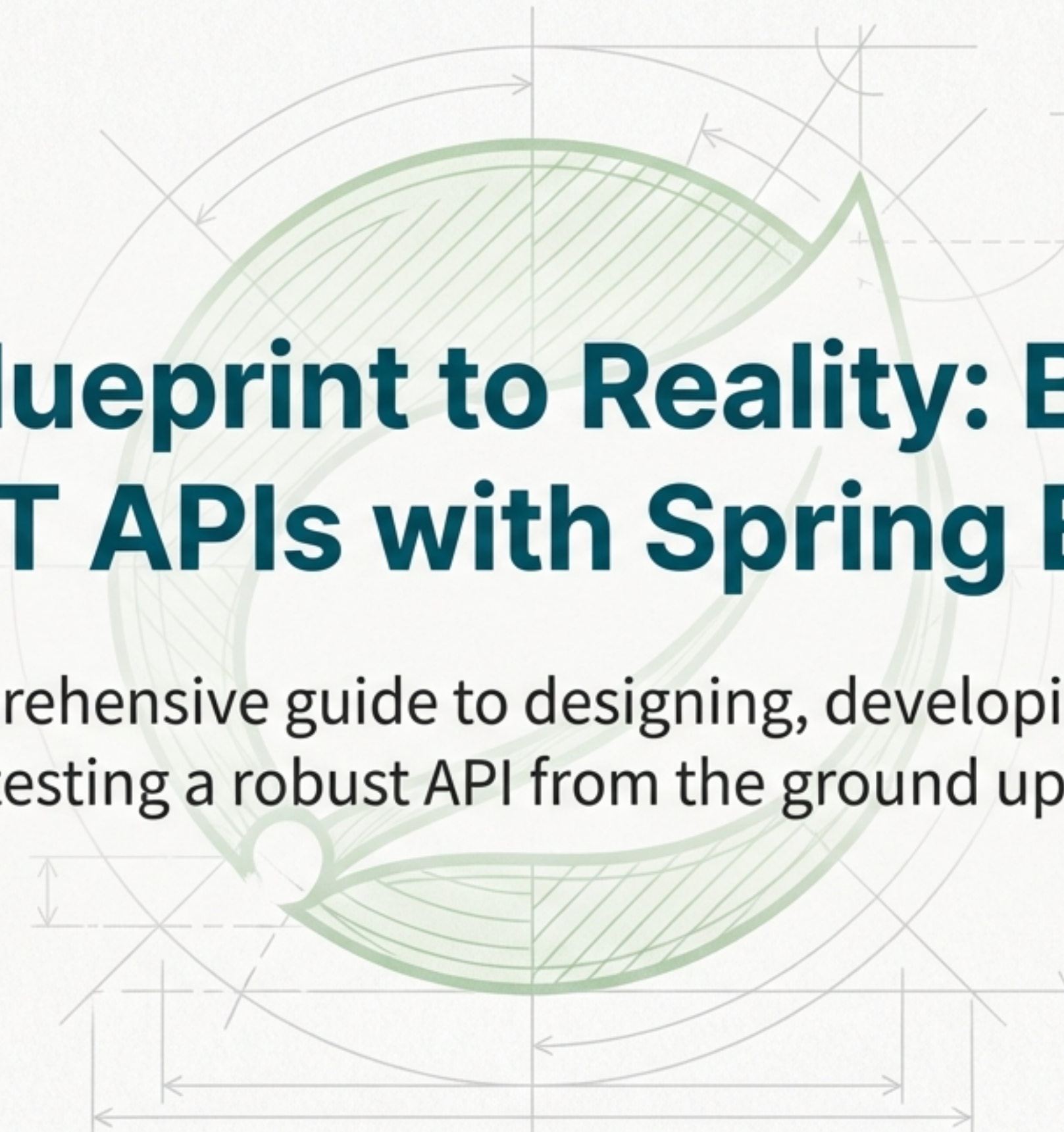


A technical drawing of a leaf, likely a magnolia leaf, shown in profile. The leaf is green with dark green veins. It is surrounded by a complex network of light gray construction lines, including arcs, straight lines, and dimension lines with arrows, suggesting a precise geometric or engineering drawing. The drawing is centered on the page.

From Blueprint to Reality: Building REST APIs with Spring Boot

A comprehensive guide to designing, developing, and testing a robust API from the ground up.

The Anatomy of a Modern Web Application

Professional applications aren't built as a single block of code. They are organized into distinct layers, each with a specific responsibility. This "separation of concerns" makes the application robust, scalable, and easier to maintain.



Presentation Layer

The entry point. Handles all incoming requests from clients (browsers, mobile apps) and routes them appropriately.



Business Logic Layer

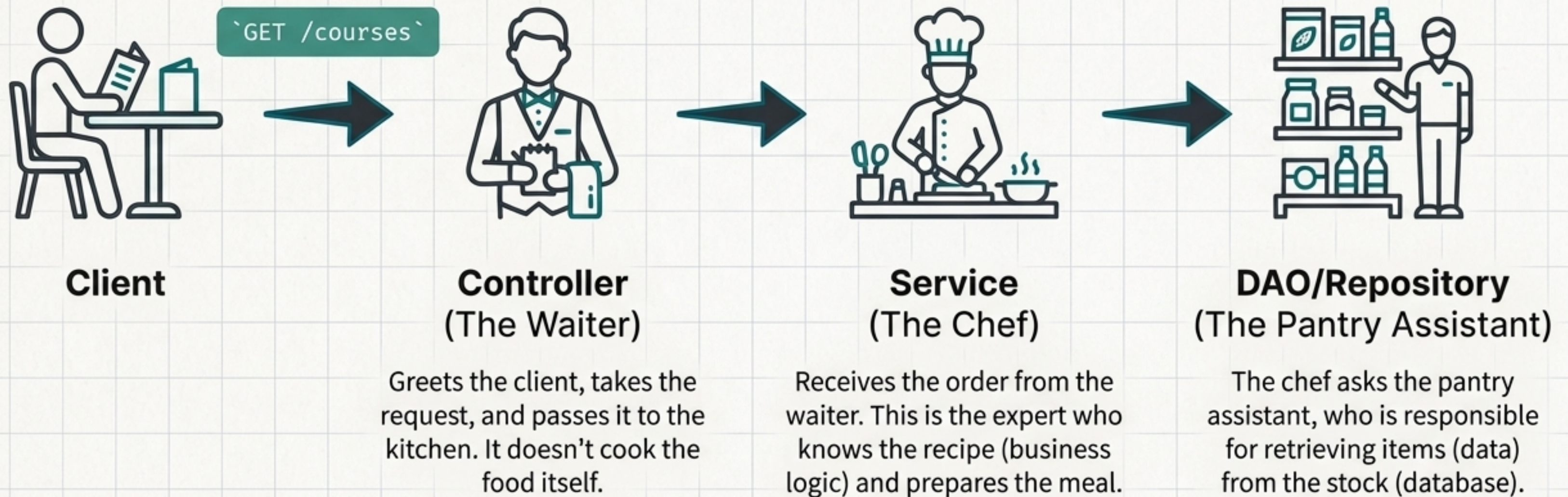
The core of the application. Executes the main business logic and coordinates tasks. This is where the 'thinking' happens.



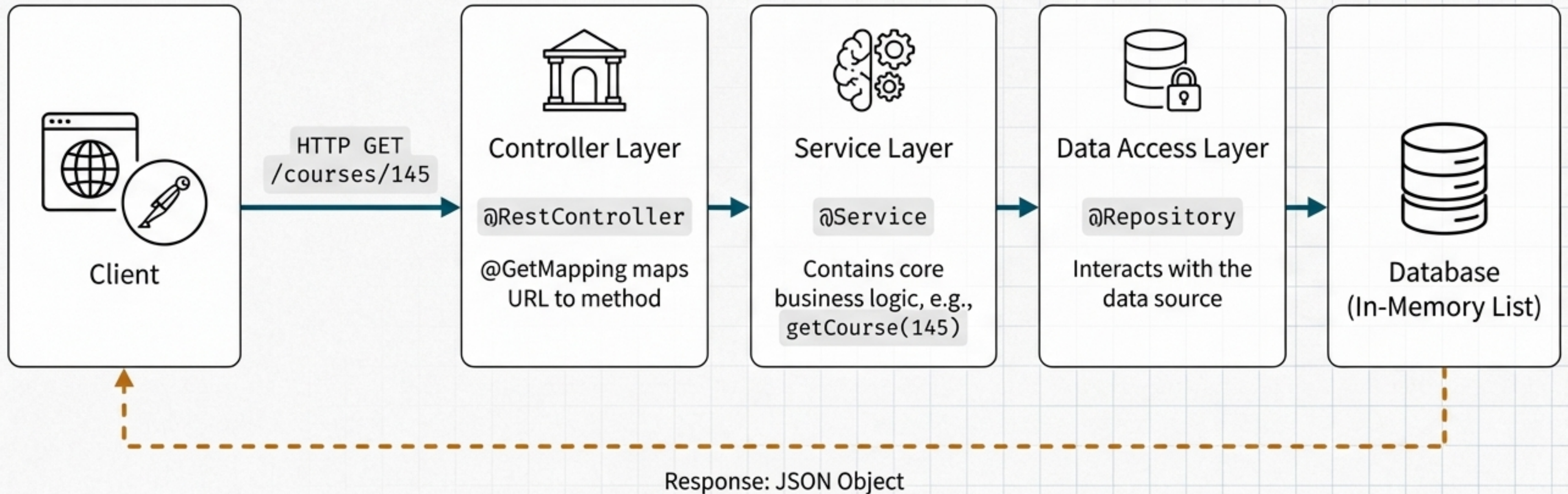
Data Access Layer

The guardian of the data. Manages all communication with the database, handling data storage and retrieval.

Think of Your Application as a Restaurant



The Journey of a Request



Our API Blueprint: A Course Management System

We will build a REST API to manage a list of courses. The API will support creating, retrieving, updating, and deleting courses. The following table defines the contract for our API.

HTTP Method	URL Path	Operation Description
GET	/courses	Retrieve a list of all available courses.
GET	/courses/{courseId}	Retrieve a single course by its unique ID.
POST	/courses	Add a new course to the list.
PUT	/courses	Update an existing course.
DELETE	/courses/{courseId}	Delete a course by its unique ID.

Step 1: Laying the Foundation with Spring Initializr

Spring Initializr (start.spring.io) is a web tool that rapidly generates a Spring Boot project structure, so you can focus on writing code, not on configuration.

Project

Language

Spring Boot Version

Maven

Java

2.7.5

Project Metadata

Group

com.springrest

Artifact

springrest

Packaging

Jar

Java

17

Dependencies

Search...

✓

Spring Web
Essential for building web applications, including RESTful APIs.

✓

Spring Data JPA
To persist data in SQL stores...

✓

MySQL Driver
The JDBC driver for MySQL.

Step 2: Defining Our Data Model (The `Entity`)

Before we can manage courses, we need to define what a 'Course' is. We create a simple Java class (a POJO - Plain Old Java Object) to represent our data structure.

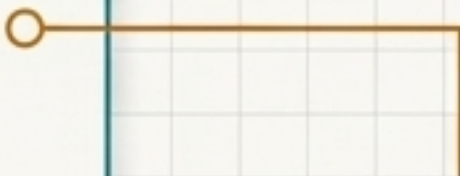
Course.java

```
// Package: com.springrest.springrest.entities

public class Course {
    private long id;
    private String title;
    private String description;

    // Constructors (Default & Parameterized)
    public Course(long id, String title, String description) { ... }
    public Course() { ... }

    // Getters and Setters
    public long getId() { ... }
    public void setId(long id) { ... }
    // ... for title and description
}
```



This class is a simple blueprint. It holds the data but contains no business logic.

Step 3: Building the Business Logic (The `Service`)

The Service Layer is where the main application logic resides. We'll follow a best practice: define a `CourseService` interface first, then create an implementation class. This promotes 'loose coupling,' making our code more flexible and testable.

CourseService.java

```
// Package: com.springrest.springrest.services

import com.springrest.springrest.entities.Course;
import java.util.List;

public interface CourseService {
    public List<Course> getCourses();
    public Course getCourse(long courseId);
    public Course addCourse(Course course);
}
```

The Implementation

Next, we'll create a class that *implements* this interface and contains the actual logic.

Implementing the `CourseService`

The `CourseServiceImpl` class provides the concrete logic for the methods defined in our interface. For now, we will use a simple in-memory `List` to store our courses.

CourseServiceImpl.java

```
// Package: com.springrest.springrest.services

@Service // Marks this class as a Spring Service component
public class CourseServiceImpl implements CourseService {
    List<Course> list;

    public CourseServiceImpl() {
        list = new ArrayList<>();
        list.add(new Course(145, "Java Core Course", "Basics of Java"));
        list.add(new Course(4343, "Spring Boot Course", "Creating REST API"));
    }

    @Override
    public List<Course> getCourses() {
        return list;
    }

    // ... implementations for getCourse and addCourse ...
}
```

● This annotation tells Spring to create and manage an instance (a 'bean') of this class, making it available for other parts of our application to use.

Step 4: Creating the API Gateway (The `Controller`)

The Controller's job is to accept incoming HTTP requests, use the Service to perform the required action, and return a response. Spring Boot uses powerful annotations to make this simple.

MyController.java

```
// Package: com.springrest.springrest.controller

@RestController
public class MyController {

    @Autowired
    private CourseService courseService;

    // Get all courses
    @GetMapping("/courses")
    public List<Course> getCourses() {
        return this.courseService.getCourses();
    }
}
```

@RestController in Inter SemiBold

Tells Spring this class handles HTTP requests and that method return values should be written directly to the response body as JSON.

@GetMapping("/courses")

Maps HTTP GET requests for the `/courses` URL to this specific method.

Connecting the Layers with @Autowired

How does the `Controller` get access to the `Service`? We don't create it manually with `new CourseServiceImpl()`. Instead, we let the Spring framework 'inject' the service dependency for us.

This is a core principle called Dependency Injection.



MyController.java

```
@RestController
public class MyController {

    @Autowired // The magic glue!
    private CourseService courseService;

    // ... methods
}
```

The `@Autowired` annotation tells Spring: "Find the managed bean that implements `CourseService` (our `CourseServiceImpl`) and automatically assign it to this field." This decouples our components.

Handling Dynamic URLs with @PathVariable

Our API needs to retrieve a single course using its ID (e.g., /courses/145). We use the `@PathVariable` annotation to extract this dynamic value from the URL and use it as a method parameter.

Code Spotlight

MyController.java

```
// Get single course by ID
@GetMapping("/courses/{courseId}")
public Course getCourse(@PathVariable String courseId) {
    return this.courseService.getCourse(Long.parseLong(courseId));
}
```

A dashed orange arrow originates from the `{courseId}` placeholder in the `@GetMapping` URL and points to the `@PathVariable` annotation. Another dashed orange arrow points from the `@PathVariable` annotation to the `courseId` parameter in the method signature.

Binds the value of the URI template variable `{courseId}` to the `courseId` method parameter. Whatever ID the client puts in the URL is now available in our code.

Accepting Data with @RequestBody

To add a new course, the client sends the course data (as JSON) in the body of a POST request. The `@RequestBody` annotation automatically deserializes this JSON into our `Course` Java object.

```
{
  "id": 4,
  "title": "New Course",
  "description": "A new
course description"
}
```

POST Request Body

MyController.java

```
// Add a new course
@PostMapping("/courses")
public Course addCourse(@RequestBody Course course) {
    return this.courseService.addCourse(course);
}
```

@PostMapping("/courses")

Maps HTTP POST requests for `/courses`` to this method, used for creating new resources.

@RequestBody

Tells Spring to take the JSON from the request's body and convert it into a `Course`` object.

The Proof: Testing Our API with Postman

With the code in place, we can run our Spring Boot application and use a tool like Postman to send requests and verify the responses.

Get All Courses



Get Single Course



Add a New Course



From Blueprint to a Working API

You have successfully built a fully functional, three-layer REST API using Spring Boot. You've learned to:

- ✓ Structure an application with Controllers, Services, and Entities.
- ✓ Define an API contract with clear endpoints.
- ✓ Use key Spring annotations (`@RestController`, `@GetMapping`, `@Autowired`) to bring the API to life.
- ✓ Test your endpoints with Postman.

The Path Forward: What's Next?

Connecting to a Real Database

Our current service uses a temporary in-memory list. The next evolution of this project is to replace that list by **connecting our application to a persistent database (like MySQL) using Spring Data JPA and Hibernate**. This will allow our data to **survive application restarts**.

